

$F2 \parallel wC_{\max}$ Structural Properties and Dynamic Programming

Author One^{a,*}, Author Two^b

^aInstitution One, Location, Country

^bInstitution Two, Location, Country

Abstract

Minimizing $wC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel wC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π . Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1998) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $wC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel wC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \dots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on

*Corresponding author. E-mail address: author@email.com

M_1 for a_j time units and then on M_2 for b_j time units, with no preemption allowed. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ specifies the processing order on both machines. For a schedule π , let $A_j = \sum_{h=1}^j a_{\pi(h)}$ be the cumulative M_1 completion time after the first j jobs. The makespan $C_{\max}(\pi)$ is the M_2 completion time of the last job, defined recursively by

$$C_j = \max\{C_{j-1}, A_j\} + b_j, \quad j = 1, \dots, n, \quad (1)$$

and $C_0 = 0$, $C_{\max}(\pi) = C_n$. The objective is to find a schedule π that minimizes $wC_{\max}$.

For $F2 \parallel wC_{\max}$, a permutation schedule in which the job order on M_1 and M_2 coincides is always optimal (Baker, 1974). Johnson's rule (Johnson, 1954) sequences all n jobs to minimize the makespan, yielding a global order $\pi = (\pi(1), \pi(2), \dots, \pi(n))$.

Lemma 2.1. *For $F2 \parallel wC_{\max}$, there exists an optimal schedule in which all jobs are sequenced according to Johnson's rule.*

Proof. Consider an optimal schedule π in which two adjacent jobs J_i and J_j violate Johnson's rule, i.e., J_i immediately precedes J_j but $\min\{a_i, b_j\} > \min\{a_j, b_i\}$. Swapping them yields a schedule π' . By exchange, the completion times of the two jobs do not increase under π' , and M_2 becomes free for the next job no later than before. Hence no job's completion time increases. Consequently $\max_j w_j C_j(\pi') \leq \max_j w_j C_j(\pi)$. Repeated adjacent swaps therefore transform any optimal schedule into one that follows Johnson's rule without degrading the objective. $\square$

Theorem 2.1. *$F2 \parallel wC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \dots, x_r$ with $\sum_{i=1}^r x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \dots, r\}$ such that $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T$?

Construct an instance of $F2 \parallel wC_{\max}$. Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \dots, n - 1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n . Does there exist a threshold Y such that:

$$wC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

We partition jobs j_1 to j_{n-1} into two subsets via j_n , the distribution of jobs on the machine is $(\mathcal{J}_{j \in A_1}, J_n, \mathcal{J}_{j \in A_2})$. Since the processing times of A_1 and A_2 are unknown, while their total processing time is $2T$. We may assume that the processing time of A_1 on M_2 is $r(T + \xi)$, A_2 is $r(T - \xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \quad (2)$$

$$C_n^2 = 1 + r(T + \xi) + 1 \quad (3)$$

and $C_n = \max(C_n^1, C_n^2)$

If $\xi = 0$, then $\sum_{j \in A_1} b_j = \sum_{j \in A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (2), we have:

$$\begin{aligned} w_n C_n &= (2T + 1)(|A_1| + rT + 1) \\ &\leq (2T + 1)(rT + r) \\ &= r(T + 1)(2T + 1) \\ &= Y \\ w C_{\max} &= r(T + 1)(C_n + rT) = Y \end{aligned}$$

so $w_j C_j \leq w C_{\max} = Y$

If the scheduling problem exists a threshold y such that $w C_{\max} \leq y$. When $\xi > 0$, C_n is determined by (3), we have:

$$\begin{aligned} w_n C_n &= (1 + r(T + \xi) + 1)(2T + 1) \\ &\geq (1 + r(T + 1) + 1)(2T + 1) \\ &> Y \end{aligned}$$

When $\xi < 0$, C_n is determined by (2), we have:

$$\begin{aligned} w C_{\max} &= (C_n + \sum_{j \in A_2} b_j)(T + 1) \\ &= (1 + rT + |A_1| + r(T - \xi))(T + 1) \\ &\geq (rT + r + rT)(T + 1) \\ &= r(T + 1)(2T + 1) = Y \end{aligned}$$

so $w C_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel w C_{\max}$ instance admits a schedule with $w C_{\max} \leq Y$. Since Partition is NP-complete, $F2 \parallel w C_{\max}$ is NP-hard. $\square$

3 $F2 \parallel w C_{\max}$ for the number of weights is constant

When the number of distinct weights K is bounded by a constant, the structural properties established in Section 2 enable a polynomial-time dynamic programming algorithm. By Lemma 2.1, the initial job sequence follows Johnson's rule; the block partition and all subsequent steps are defined with respect to this Johnson order. We first describe the block partition and group logic underlying our approach, then present the DP formulation, and finally give the complete algorithm and its complexity analysis.

3.1 Algorithm Properties and Dynamic Programming

The jobs are partitioned into blocks by a single left-to-right scan. Starting from the first job, a new block is created whenever a weight value not encountered before appears; that job becomes the final job of the current block, and the next job starts a new block. Since each block introduces at least one previously unseen weight, the procedure yields at most K blocks, denoted $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K$. By construction, the final job of each block carries the weight that triggered the cut. By construction, block weights are non-increasing, i.e., $W(\mathcal{B}_1) \geq W(\mathcal{B}_2) \geq \dots \geq W(\mathcal{B}_K)$. Moreover, in any optimal schedule, jobs belonging to $\mathcal{B}_1 \cup \dots \cup \mathcal{B}_k$ must be scheduled before any job from $\mathcal{B}_{k+1} \cup \dots \cup \mathcal{B}_K$.

Jobs of the same weight form a natural group, denoted $\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_K$. Scheduling heavier groups before lighter ones automatically enforces the non-increasing block-weight condition, and scanning groups in descending weight order directly yields the block partition.

Example 3.1. Consider a 6-job instance with Johnson sequence $\pi = (J_1, \dots, J_6)$ and weights $(w_1, \dots, w_6) = (2, 2, 5, 2, 4, 2)$. Scanning π from left to right and cutting whenever a weight not seen before appears yields the following partition:

$$\underbrace{J_1, J_2, J_3}_{\mathcal{B}_1} \quad \underbrace{J_4, J_5}_{\mathcal{B}_2} \quad \underbrace{J_6}_{\mathcal{B}_3}$$

The resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, $\mathcal{B}_3 = \{J_6\}$. And $\mathcal{G}_1 = \{J_1, J_2, J_4, J_6\}$, $\mathcal{G}_2 = \{J_5\}$, $\mathcal{G}_3 = \{J_3\}$.

3.1.1 Dynamic Programming Formulation

We now develop a dynamic programming algorithm that constructs the optimal schedule block by block. Our state representation adapts the interval-based framework of (Hall, 1998), originally devised for $F2|r_j|C_{\max}$, to the $wC_{\max}$ objective.

State Definition. Let $\mathcal{H}$ denote the state of K blocks, represented as

$$\mathcal{H} = (\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K),$$

where each block $\mathcal{B}_k$ is described by a 4-tuple for job J_j , and by $L_{(k)}$ record the weight of the last job in each block :

$$\mathcal{B}_k = (A_j, B_j, C_j, L_{(k)}).$$

The components are defined as follows:

- A_j — Calculate the cumulative processing time on machine M_1 ;
- B_j — Calculate the cumulative processing time on machine M_2 ;

- C_j — Calculate the makespan of block k , computed via the recurrence of equation (1);
- $L_{(k)}$ — the weight of the last job in block k .

The initial state corresponds to an empty schedule: $\mathcal{H}_0 = (\emptyset, \dots, \emptyset)$ where each empty block is $(0, 0, 0, \emptyset)$.

State Transition. Given a state $\mathcal{H}$ after processing $k-1$ blocks for $\mathcal{J} = \{J_1, J_2, \dots, J_n\}$, We append the next block $\mathcal{B}_k$ and J_j is next job.

$$A_j = A_{j-1} + a_j, \quad (4)$$

$$B_j = B_{j-1} + b_j, \quad (5)$$

$$C_j = \max(C_{j-1}, A_j) + b_j. \quad (6)$$

After all jobs in $\mathcal{B}_k$ have been processed, $L_{(k)}$ is set to $W(\mathcal{B}_k)$, the weight of the final job in block k , and the block 4-tuple is finalized as

$$\mathcal{B}_k = (A, B, C, L_{(k)}) \quad (7)$$

Dominance Lemma.

Lemma 3.1. *Let $\mathcal{H} = (\mathcal{B}_1, \dots, \mathcal{B}_k)$ and $\mathcal{H}' = (\mathcal{B}'_1, \dots, \mathcal{B}'_k)$ be two states after processing the same set of k blocks. If $C_{(k)} \leq C'_{(k)}$ and $L_{(k)} \leq L'_{(k)}$, then $\mathcal{H}'$ is dominated and can be removed from further consideration.*

Proof. Let π and π' be the partial schedules corresponding to $\mathcal{H}$ and $\mathcal{H}'$. For any suffix schedule σ of the remaining jobs, the makespan contribution is monotone non-decreasing in the prefix completion times. The condition $C_{(k)} \leq C'_{(k)}$ ensures that π leads to no larger makespan than π' under any suffix. Moreover, $L_{(k)} \leq L'_{(k)}$ guarantees that the maximum weighted completion time among already scheduled jobs is no larger under π . Hence π dominates π' under the $wC_{\max}$ objective. $\square$

The dominance rule of Lemma 3.1 justifies retaining only Pareto-optimal states with respect to $(C_{(k)}, L_{(k)})$ after processing each block. This pruning is applied at the job level within each block, as each job is appended, to minimize the number of active states.

3.1.2 Algorithm 1

The complete DP procedure is presented as three algorithms below.

Block Partitioning

- 1: **Input:** The Johnson sequence π of $\mathcal{J}$; weights $w_j, j = 1, \dots, n$.
- 2: **Output:** Blocks $\mathcal{B}_1, \dots, \mathcal{B}_K$ and their weights $W(\mathcal{B}_k)$.
- 3: $S \leftarrow \{w_{\pi(1)}\}$
- 4: $k \leftarrow 1; \mathcal{B}_k \leftarrow \emptyset$
- 5: **for** $i = 1$ to n **do**
- 6: $j \leftarrow \pi(i)$
- 7: $\mathcal{B}_k \leftarrow \mathcal{B}_k \cup \{j\}$
- 8: **if** $w_j \notin S$ **then**
- 9: $S \leftarrow S \cup \{w_j\}$
- 10: $W(\mathcal{B}_k) \leftarrow w_j$
- 11: $k \leftarrow k + 1; \mathcal{B}_k \leftarrow \emptyset$
- 12: **end if**
- 13: **end for**
- 14: **if** $\mathcal{B}_k = \emptyset$ **then**
- 15: $K \leftarrow k - 1$
- 16: **else**
- 17: $W(\mathcal{B}_k) \leftarrow w_{\pi(n)}$
- 18: $K \leftarrow k$
- 19: **end if**
- 20: **return** $\{\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K\}$

Completion-time

- 1: **Input:** Block number K ; The job set of the first k blocks $\mathcal{J}$ with (a_j, b_j, w_j) ; state $\mathcal{H}$.
- 2: **Output:** Updated state set after processing $\mathcal{B}_k$.
- 3: **for** $k = 1$ to K **do**
- 4: $\mathcal{J}_k \leftarrow \mathcal{B}_k;$
- 5: **for** each job $j \in \mathcal{J}_k$ **do**
- 6: $A_j = A_{j-1} + a_j$
- 7: $B_j = B_{j-1} + b_j$
- 8: $C_j = \max(C_{j-1}, A_j) + b_j$
- 9: **end for**
- 10: **end for**
- 11: **return** (A, B, C)

dynamic programming

- 1: **Input:** Jobs $\mathcal{J}$ with $(a_j, b_j, w_j), j = 1, \dots, n$.

```

2: Output: The optimal makespan  $C_{\max}$ .
3:  $\{\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K\} \leftarrow \text{Block Partitioning}(\pi)$ 
4:  $\mathcal{H} \leftarrow \{(0, 0, 0, 0)\}$ 
5: for  $k = 1$  to  $K$  do
6:    $\mathcal{H} \leftarrow \text{Completion-time}(k, \mathcal{H})$ 
7:   for each state  $s \in \mathcal{H}$  do
8:      $L_{(k)} \leftarrow W(\mathcal{B}_k)$ 
9:   end for
10: end for
11:  $C_{\max} \leftarrow \min\{C \mid (A, B, C, L) \in \mathcal{H}\}$ 
12: return  $C_{\max}$ 

```

3.1.3 Complexity Analysis

Theorem 3.1. *Algorithm 1 computes an schedule for $F2 \parallel wC_{\max}$ in $O()$ time.*

Proof.

□

3.2 FPTAS via Geometric Rounding

4 Conclusion

References

- Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.
- Hall, L. A. (1998). Approximability of flow shop scheduling. *Mathematical Programming*, 82(1–2):175–190.
- Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.